Pokhara University
Nepal Engineering College, Changunarayan, Bhaktapur

Software Engineering and Project Management

Presented by: Er. Susant Bhujel

Department of Computer Science and Engineering

Introduction

Software

- Software is a set of instructions or programs that tells a computer what to do.
- Software is collection of computer programs and associated documentation such as requirements, design models and user manuals.
- Software products may be developed for a particular customer or may be developed for a general market.
- New software can be created by developing new programs, configuring generic software systems or reusing existing software.
- Software products may be:
  - Generic - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.
  - Bespoke (custom) - developed for a single customer according to their specification. E.g. esewa, Khalti, etc.

Nature and Characteristics of Software

The **nature of software** refers to its key characteristics, which differentiate it from other products or technologies.

According to Fred Brooks, the nature of software are:

1. Complexity
2. Conformity
3. Changeability
4. Invisibility

Nature and Characteristics of Software (Contd…)

Complexity:

- Brooks highlighted that software systems are inherently complex, far more so than most other human-made systems.
- This complexity arises because software interacts with a wide variety of hardware components, users, environments, and other software systems.
- This complexity makes the software difficult to understand, design, test, and maintain.
- Complexity can hide defects that may not be discovered easily, thus requiring significant additional and unplanned rework.

Conformity:

- Software must conform to the arbitrary rules and constraints of the environment in which it operates, such as existing hardware, user interfaces, business rules, and external systems
- Unlike physical artifacts, which are shaped by predictable natural laws, software often conforms to the unpredictable decisions of humans, making it difficult to standardize.

Nature and Characteristics of Software (Contd…)

Changeability:

- Software is more susceptible to change than physical systems.
- Brooks observed that software constantly needs to be updated or modified to accommodate new requirements, bug fixes, platform changes, or improvements.
- The malleability of software leads to continuous and sometimes unexpected changes, complicating its development and maintenance.

Invisibility:

- Unlike physical products, software has no inherent visual representation.
- Its structure and behavior are abstract, making it difficult to visualize, design, or communicate about it.
- Diagrams or flowcharts only partially represent the underlying complexity, leading to challenges in understanding how software functions as a whole.
- While the effects of executing software on a digital computer are observable, software itself cannot be seen, tasted, smelled, touched, or heard. Software is an intangible entity because our five human senses are incapable of directly sensing it.

Software versus System Engineering

Software Engineering:

- Focus on the development, maintenance, and improvement of software applications. They work on the entire software development life cycle, from gathering requirements to testing and maintenance. Software engineers need to be proficient in programming languages, software development tools, and operating systems.

System Engineering:

- Focus on the design, integration, and maintenance of complex systems, including computer networks. They work on the entire project engineering life cycle, and often have more experience working with hardware and networks. Systems engineers are responsible for ensuring that systems work seamlessly.

Software Crisis

- Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs as per the requirements in the required time.
- The **software crisis** refers to a set of challenges faced by the software industry, particularly during the 1960s and 1970s, due to the rapid increase in the complexity and demand for software, which outpaced the ability to manage, develop, and maintain it effectively.
- This term was coined when it became evident that traditional engineering methods were inadequate for producing reliable, efficient, and maintainable software at scale.
- The crisis highlighted a range of issues, including budget overruns, missed deadlines, poor quality, and even complete project failures.

Software Crisis

- The causes of the software crisis were linked to the overall complexity of hardware and the software development process. The crisis manifested itself in several ways:
  - Projects running over-budget
  - Projects running over-time
  - Software was very inefficient
  - Software was of low quality
  - Software often did not meet requirements
  - Projects were unmanageable and code difficult to maintain
  - Software was never delivered
- Various processes and methodologies have been developed over the last few decades to improve software quality management such as procedural programming and object-oriented programming. However, software projects that are large, complicated, poorly specified, or involve unfamiliar aspects, are still vulnerable to large, unanticipated problems.

Software Myths

- **Software myths** are misconceptions or false beliefs about software development that can lead to unrealistic expectations, poor planning, and project failures.
- These myths are often held by developers, project managers, clients, or other stakeholders, and they tend to oversimplify or misunderstand the complexities of software engineering.
- Types of Software Myths:

I. Management Myths

II. Customer Myths

III. Practitioner's Myths

Software Myths (contd…)

Management Myths:

Myth 1: We already have a book that's full of standards and procedures for building software. Won't that provide my people with everything they need to know?

Reality:

- Software experts do not know all the requirements for the software development.
- And all existing processes are incomplete as new software development is based on new and different problem.

Software Myths (contd…)

Myth 2: If we get behind schedule, we can add more programmers and catch up

Reality:

- Software development is not a mechanistic process like manufacturing. In the words of Brooks:" adding people to a late software project makes it later." At first, this statement may seem counterintuitive. However, as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

Myth 3: If I decide to outsource the software project to a third party, I can just relax and let that firm build it.

Reality:

- If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

Software Myths (contd…)

Customer Myths

Myth 1: A general statement of objectives is sufficient to begin writing programs-we can fill in the details later.

Reality:

- Although a comprehensive and stable statement of requirements is not always possible, an ambiguous “ statement of objectives” is a recipe for disaster.
- Unambiguous requirements (usually derived iteratively) are developed only through effective and continuous communication between customer and developer.

Myth 2: Software requirements continually change, but change can be easily accommodated because software is flexible.

Software Myths (contd…)

Practitioner's Myths:

Myth 1: Once we write the program and get it to work, our job is done.

Myth 2: Until I get the program running, I have no way of assessing its quality.

Myth 3: The only deliverable work product for a successful project is the working program.

Myth 4: Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

Four Ps of Software Project Management

- The **4 Ps of Software Project Management** are key elements that contribute to the success of a software project.
- They are **People, Product, Process, and Project**. Each "P" represents a fundamental aspect of managing and executing a software project effectively.

Four Ps of Software Project Management (contd…)

People:

The most important component of a product and its successful implementation is human resources. In building a proper product, a well-managed team with clear-cut roles defined for each person/team will lead to the success of the product. We need to have a good team in order to save our time, cost, and effort. Some assigned roles in software project planning are **project manager, team leaders, stakeholders, analysts,** and other **IT professionals**. Managing people successfully is a tricky process which a good project manager can do.

Product:

As the name inferred, this is the deliverable or the result of the project. The project manager should clearly define the product scope to ensure a successful result, control the team members, as well technical hurdles that he or she may encounter during the building of a product. The product can consist of both tangible or intangible such as shifting the company to a new place or getting a new software in a company.

Four Ps of Software Project Management (contd…)

Process:

In every planning, a clearly defined process is the key to the success of any product. It regulates how the team will go about its development in the respective time period. The Process has several steps involved like, documentation phase, implementation phase, deployment phase, and interaction phase.

Project:

The last and final P in software project planning is Project. It can also be considered as a blueprint of process. In this phase, the project manager plays a critical role. They are responsible to guide the team members to achieve the project's target and objectives, helping & assisting them with issues, checking on cost and budget, and making sure that the project stays on track with the given deadlines.

Software Measurement and Metrics

- **Software measurement** is the process of assigning numbers or values to attributes of software to quantify various aspects of its development, performance, quality, and usability.

Software Measurement Principles

- The software measurement process can be characterized by five activities-

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.

2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.

3. **Analysis:** The computation of metrics and the application of mathematical tools.

4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.

5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Software Measurement and Metrics (contd…)

Need for Software Measurement

Software is measured to:

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.
- Enable data-driven decision-making in project planning and control.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Give software products and processes a quantitative basis for evaluation.
- Enable the ongoing improvement of software development practices.

Software Measurement and Metrics (contd…)

Type of Software Measurements

1. Direct Measurement

- Involves quantifying attributes that can be observed directly.
- **Examples**:
  - Lines of Code (LOC): Measures the size of the software by counting code lines.
  - Execution Time: Measures the time required to execute specific operations.
  - Number of Defects: Counts the defects identified in a given unit of code or over time.

2.Indirect Measurement

- Involves quantifying attributes derived from one or more direct measurements.
- **Examples**:
  - Code Quality: Derived from metrics like defect density or cyclomatic complexity.
  - Maintainability: Assessed using factors like code modularity, cyclomatic complexity, and reusability.
  - Productivity: Calculated by dividing total code produced (e.g., LOC) by the effort or time spent on development

Software Measurement and Metrics (contd…)

- **Software metrics** are quantitative measures that provide insights into the development, quality, and performance of software.
- They help in evaluating the effectiveness of processes, tracking project progress, assessing code quality, and ensuring the product meets desired standards.
- Metrics support data-driven decision-making and enable improvements in various stages of software engineering.

There are 4 functions related to software metrics:

I. Planning
II. Organizing
III. Controlling
IV. Improving

Software Measurement and Metrics (contd…)

Characteristics of software Metrics

1. **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.

2. **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.

3. **Applicability:** Metrics should be applicable in the initial phases of the development of the software.

4. **Repeatable:** When measured repeatedly, the metric values should be the same and consistent.

5. **Economical:** The computation of metrics should be economical.

6. **Language Independent:** Metrics should not depend on any programming language.

Software Measurement and Metrics (contd…)

Types of Software Metrics

I. Product Metrics
II. Process Metrics
III. Project Metrics

Product Metrics:

- Metrics that measure the characteristics of the software product itself, including its quality, complexity, and size.
- Some common product metrics are:

I. Fan-in/Fan-out
II. Length of Code
III. Cyclomatic complexity
IV. Length of Identifiers
V. Depth of conditional nesting
VI. Fog index

Software Measurement and Metrics (contd…)

Fan-in:

- The number of modules or components that directly call or refer to a particular module.
- **Significance**:
  - High fan-in indicates that a module is widely reused, which can be a good sign of modularity and code reuse.
  - However, excessive fan-in can make the module critical to the system, making it a potential bottleneck or single point of failure.
- Example: module A is called by modules B,C and D. Fan-in of A=3.

Fan-out:

- The number of modules or components that a particular module directly calls or interacts with.
- **Significance**:
  - High fan-out indicates that a module is highly dependent on many other modules.
  - A high fan-out can make the module difficult to test, maintain, and modify because changes in the called modules may affect it.
- Example: module X directly calls modules Y, Z, and W. Fan-out of X=3.

Software Measurement and Metrics (contd…)

Line of code:

- **Lines of Code (LOC)** is a software metric used to measure the size or complexity of a program by counting the number of lines in its source code. It is one of the simplest and most widely used metrics for software development.

Type of LOC:

Physical LOC:

- Counts all lines in the source code, including comments, blank lines, and code lines.

Logical LOC:

- Counts only executable lines of code, ignoring blank lines and comments.

Software Measurement and Metrics (contd…)

Importance of LOC

Effort Estimation:

- Provides a rough idea of the development effort required for the project.
- More LOC often means more effort and time for development and maintenance.

Productivity Measurement:

- LOC can be used to measure developer productivity (e.g., LOC per day)

Software Quality:

- Helps assess the complexity of a program. Fewer lines with high functionality often indicate better quality.

Cost Estimation:

- Used in models like **COCOMO (Constructive Cost Model)** for project cost estimation.

Software Measurement and Metrics (contd…)

Cyclomatic Complexity:

- **Cyclomatic Complexity** is a software metric used to measure the complexity of a program's control flow. It quantifies the number of independent paths through the source code, helping assess the complexity, maintainability, and testability of a program.
- It is computed using the Control Flow Graph of the program.
- **Nodes(N)**: Represent statements or blocks of code.
- **Edges(E)**: Represent the control flow between statements
- **Formula**: The formula to calculate cyclomatic complexity is:
  - Cyclomatic Complexity (V)=E−N+2P
  - P: Number of connected components or exit points (usually P=1 for a single program).

Software Measurement and Metrics (contd…)

Length of identifier:

- Length of Identifiers is a measure of the average length of distinct identifier in a program.
- The longer the identifiers, the more understandable the program.
- The length of identifiers metric helps in assessing the readability and maintainability of the code.
- Longer, descriptive names generally indicate better readability and understanding.

Depth of Conditional Nesting:

Depth of Nesting Condition is a measure of the depth of nesting of if statements in a program.

Deeply nested statements are hard to understand and are potentially error-prone.

It indicates how many levels deep the nested conditional statements (such as if, else, while, for, etc.) go.

This metric is important because deeply nested code can be difficult to read, understand, and maintain.

Software Measurement and Metrics (contd…)

Fog index:

- Fog Index is a measure of the average length of words and sentences in documents.
- The higher the value for the Fog index, the more difficult the document may be to understand.
- Formula:

  Fog Index = 0.4 × (Number of Words / Number of Sentences + 100×Number of Complex Words / Number of Words)

  where:

  - ***Number of Words**: Total words in the text.*
  - ***Number of Sentences**: Total sentences in the text.*
  - ***Number of Complex Words**: Words with three or more syllables, not including proper nouns, familiar jargon, or compound words.*

Software Measurement and Metrics (contd…)

Process Metrics:

- Metrics focused on the software development process, assessing the efficiency and effectiveness of different activities

Some common Process Metrics are:

I. Lead time
II. Cycle time
III. Work in Progress
IV. Process Efficiency
V. Process Compliance

Software Measurement and Metrics (contd…)

Lead Time:

- Lead time refers to the time from when a development task is requested or a new feature is planned until it is completed and deployed.
- **Components of Lead Time in Software Development**
  1. **Development Time**: Time spent writing and testing the code.
  2. **Approval Time**: Time spent waiting for approvals from stakeholders or managers.
  3. **Deployment Time**: Time taken to deploy and release the software to users.
- **Lead time can be impacted by factors like:**
  - **Team Efficiency**: How quickly the team can work through tasks.
  - **Complexity of Tasks**: More complex tasks naturally require more time.
  - **Queue Time**: Time spent waiting for resources, feedback, or approvals.
  - **Bottlenecks**: Any stage in the process where work slows down significantly.

Software Measurement and Metrics (contd…)

Why Lead Time is Important

1. **Customer Satisfaction**: Shorter lead times mean that customers get their desired features or products faster, leading to greater satisfaction.
2. **Improved Efficiency**: Tracking lead time helps identify bottlenecks and inefficiencies in the workflow, allowing teams to improve their processes.
3. **Predictability**: By measuring lead time over multiple iterations, teams can better predict when tasks or features will be delivered in future projects.
4. **Resource Allocation**: Understanding lead time helps teams manage resources more effectively by aligning tasks with team capacity.

Software Measurement and Metrics (contd…)

Cycle Time:

- **Cycle Time** refers to the total time it takes to complete a specific task or process once it has started.
- In the context of software development, cycle time measures the duration from the start of work on a feature, bug fix, or other tasks until its completion and delivery.
- Cycle time is often used to evaluate the efficiency and performance of teams and processes, helping to identify bottlenecks or inefficiencies that may slow down work.

Why Cycle Time is Important

1. **Process Efficiency**: Tracking cycle time helps teams identify and eliminate inefficiencies in their workflow. If cycle times are long, there may be bottlenecks or delays that need to be addressed.
2. **Predictability**: Understanding cycle time helps teams predict how long it will take to complete similar tasks in future projects. It aids in planning and forecasting timelines more accurately.
3. **Continuous Improvement**: Regularly measuring and improving cycle time leads to better team performance and quicker delivery of features or bug fixes.
4. **Customer Satisfaction**: Shorter cycle times mean that features or fixes are delivered more quickly, leading to faster customer value delivery and higher satisfaction.

Software Measurement and Metrics (contd…)

Work in Progress:

- **Work in Progress (WIP)** refers to the tasks, items, or work that are currently being worked on but are not yet completed.

Process Efficiency:

- **Process Efficiency** in software engineering refers to the effectiveness with which a software development process achieves its goals with the least amount of wasted resources, time, and effort.
- It focuses on optimizing the use of resources, reducing unnecessary steps, and improving productivity without compromising the quality of the software being developed.

Process Compliance:

- **Process Compliance** refers to the degree to which a software development process adheres to predefined standards, methodologies, guidelines, or regulations that are established within an organization or industry.
- In other words, it is about ensuring that the processes followed during software development align with best practices, regulatory requirements, and internal policies.

Software Measurement and Metrics (contd…)

Project Metrics:

- Metrics specific to a software project, tracking project progress, resource utilization, cost, and adherence to timelines.

Some common Project Metrics are:

I. Effort Variance

II. Schedule Variance

Software Measurement and Metrics (contd…)

Effort Variance:

- **Effort Variance** in the context of software engineering refers to the difference between the estimated effort required to complete a project or task and the actual effort that is expended during the execution of that project or task.
- It is a key metric in project management, especially when monitoring the progress of a software development project, as it helps to track whether the project is being completed within the estimated time and resources.
- Effort variance is often used in the analysis of how well a team is performing against initial estimates and is a part of the broader concept of **project performance metrics**.
- It allows project managers to assess whether a project is underestimating or overestimating the effort needed and to adjust project planning or resource allocation accordingly.
- Effort Variance (EV)=Actual Effort (AE)−Estimated Effort (EE)
- **Actual Effort (AE)**: The total amount of work (in hours or person-days) actually spent on the project.
- **Estimated Effort (EE)**: The initial estimate of the total amount of work that was anticipated at the start of the project

Software Measurement and Metrics (contd…)

Schedule Variance:

- **Schedule Variance (SV)** is a key performance metric in project management that measures the difference between the planned progress (schedule) and the actual progress achieved by a given point in time.
- It helps to track whether a project is on schedule, ahead of schedule, or behind schedule, providing important insights for project managers to make timely adjustments and manage project risks.
- Schedule Variance (SV)=Earned Value (EV)−Planned Value (PV)

  Where:
  - **Earned Value (EV)**: The value of the work actually completed at a given time, measured in terms of the budget allocated for that work. It reflects the actual progress made compared to the planned scope.
  - **Planned Value (PV)**: The value of the work that was planned to be completed by that point in time, according to the original schedule. This represents the progress that was expected by this time.

Project Estimation

- Project estimation is a complex process that revolved around predicting the time, cost, and scope that a project requires to be deemed finished.
- Software project estimation approaches assist project managers in effectively estimating critical project parameters such as cost and scope.
- PMs can then use these estimation strategies to give clients more accurate projections as well as budget the funds and resources they'll require for a project's success.
- Software project estimation is a critical component of successful software development.
- By accurately predicting project costs, scope, and so timelines, project managers can effectively allocate resources and manage expectations.
- But in terms of software development or software engineering, it also takes the experience of the software outsourcing company, the technique they have to utilize, the process they need to follow in order to finish the project (**Software Development Life Cycle**).
- Project Estimation requires the use of complex tools & good mathematical as well as knowledge about planning.

Project Estimation (contd…)

Key elements consider during the estimation of software project:

- **Cost:** Determine the financial resources needed to complete the project.
- **Time:** Estimate the overall project duration and individual task timelines.
- **Scope:** Define the project's boundaries and deliverables.
- **Risk:** Identify potential risks and develop mitigation strategies.
- **Resources:** Assess the required human, technological, and financial resources.
- **Quality:** Determine the desired level of quality and standards to be met.

Project Estimation (contd…)

- There is no one-size-fits-all solution, as evidenced by the variety of methods available. Making a perfect forecast that addresses all potential issues is extremely difficult.
- Software development is a dynamic process in which programmers are constantly learning new technologies and making new discoveries. This has a significant impact on the estimate.

Techniques for software project estimation are:

- **Top-down estimation**
- **Bottom-up estimation**
- **Expert judgment**
- **Comparative or analogous estimation**
- **Parametric modeling**

Project Estimation (contd…)

Top-down estimation:

- **Top-down estimation** involves assigning an overall time for the project and then breaking it down into individual phases, work, and tasks based on the **work breakdown structure (WBS)**.
- This approach is useful when you have a general understanding of the project's scope and complexity.

Bottom-up estimation:

- Bottom-up estimation is the polar opposite of a top-down estimate. You begin by estimating each individual task or aspect of the project using this method.
- Then you add all of the individual estimates together to create the overall project estimate.
- This type of estimate is more accurate than the top-down approach because each activity is assessed individually.
- However, it takes longer to have a completed software estimation and requires more effort from the project manager as well as business analyst.

Project Estimation (contd…)

Expert Judgment:

- Expert judgment is one of the most widely used estimation techniques because it is quick and simple.
- To estimate projects, this method relies on the expertise and intuition of experts.
- It's most useful when you're planning a standard project that your team has previously completed or has knowledge about.
- Top-down and bottom-up estimates can both be made using expert judgment.

Comparative or Analogous Estimation:

- **Comparative or analogous estimation** uses data from similar past projects to estimate the current project. This method can be useful when there is a lack of specific data for the current project.

Project Estimation (contd…)

Parametric Model Estimation:

- **Parametric modeling** applies mathematical models to estimate project parameters based on historical data.
- This technique can be useful for projects with similar characteristics to past projects.

Empirical Estimation

- Empirical estimation is a technique or model in which empirically derived formulas are used for predicting the data that are a required and essential part of the software project planning step.
- These techniques are usually based on the data that is collected previously from a project and also based on some guesses, prior experience with the development of similar types of projects, and assumptions.
- It uses the size of the software (LOC (Line of Code) and FP (Function Point)) to estimate the effort. In this technique, an educated guess of project parameters is made.

Empirical Estimation (contd…)

Structure of Estimation Models

- A typical empirical model is derived using regression analysis on data collected from past projects.
- Overall structure of such models takes the form:

  $E=A+B*(e_v)^c$

  Where A, B, and C are empirically derived constants, E is effort in person-months, and e_v is the estimation variable (either LOC or FP).
- LOC oriented estimation models are:
  I. $E=5.2*(KLOC)^{0.91}$ Walston-Felix model
  II. $E=5.5+0.73*(KLOC)^{1.16}$ Bailey-Basili model
  III. $E=3.2*(KLOC)^{1.05}$ Boehm simple model
  IV. $E=5.288*(KLOC)^{1.047}$ Doty model for KLOC > 9

  Where, KLOC = Kilo Lines of Code

Empirical Estimation (contd…)

FP oriented estimation model are:

I. E= -91.4+0.355*FP Albrecht and Gaffney Model
II. E= -37+0.96*FP Kemerer Model
III. E= -12.88+0.405*FP Small project regression Model

Empirical Estimation (contd…)

COCOMO

- The Constructive Cost Model (COCOMO) is a software cost estimation model that helps predict the effort, cost, and schedule required for a software development project. Developed by Barry Boehm in 1981, COCOMO uses a mathematical formula based on the size of the software project, typically measured in lines of code (LOC).

- The key parameters that define the quality of any software product, which are also an outcome of COCOMO, are primarily effort and schedule:
  I. **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
  II. **Schedule:** This simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months.

COCOMO (contd…)

- In the COCOMO model, software projects are categorized into three types based on their complexity, size, and the development environment. These types are:

1. **Organic:** A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.

2. **Semi-detached**: A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environments lie in between organic and embedded. The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience better guidance and creativity.

3. **Embedded:** A software project requiring the highest level of complexity, creativity, and experience requirement falls under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

COCOMO (contd…)

Aspects	Organic	Semidetached	Embedded
Project Size	2 to 50 KLOC	50-300 KLOC	300 and above KLOC
Complexity	Low	Medium	High
Team Experience	Highly experienced	Some experienced as well as inexperienced staff	Mixed experience, includes experts
Environment	Flexible, fewer constraints	Somewhat flexible, moderate constraints	Highly rigorous, strict requirements
Effort Equation	$E = 2.4(400)^{1.05}$	$E = 3.0(400)^{1.12}$	$E = 3.6(400)^{1.20}$
Example	Simple payroll system	New system interfacing with existing systems	Flight control software

COCOMO (contd…)

There are three types of COCOMO Model:

1. Basic COCOMO Model
2. Intermediate COCOMO Model
3. Detailed COCOMO Model

Basic COCOMO Model

- The Basic COCOMO model is a straightforward way to estimate the effort needed for a software development project. It uses a simple mathematical formula to predict how many person-months of work are required based on the size of the project, measured in thousands of lines of code (KLOC).

- Basic model equation

I. Effort(E)= a*(KLOC)^{b} person-months

II. Development Time (T) = c*(E)^{d} months

III. Average staff size = E/T persons

IV. Productivity = KLOC/E KLOC/PM

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Intermediate COCOMO Model

- The **Intermediate COCOMO (Constructive Cost Model)** is a step up from the basic COCOMO model.
- It provides a more accurate effort estimation for software projects by considering the influence of various 15 **cost drivers** (or project attributes) on software development.
- The Intermediate COCOMO model incorporates **15 cost drivers** to adjust the estimated effort based on various factors that affect the software development process. These cost drivers are categorized into four groups: **Product Attributes**, **Hardware Attributes**, **Personnel Attributes**, and **Project Attributes**.
- Each Cost driver is rated for the given project environment i.e. very low, low, nominal, very high and extra high
- **Product Attributes**
  - These relate to the characteristics of the software being developed.

  I. Required Software Reliability (RELY)
  II. Database Size (DATA)
  III. Product Complexity (CPLX)

Intermediate COCOMO Model (contd…)

Hardware Attributes

- These describe the constraints imposed by the hardware on the software development.

I. Execution Time Constraint (TIME)

II. Main Storage Constraint (STOR)

III. Virtual Machine Volatility (VIRT)

IV. Turnaround Time (TURN)

Personnel Attributes

- These focus on the capabilities and experience of the team involved in the project.

I. Analyst Capability (ACAP)

II. Programmer Capability (PCAP)

III. Application Experience (AEXP)

IV. Platform Experience (PEXP)

V. Language and Tool Experience (LTEX)

Intermediate COCOMO Model (contd…)

Project Attributes

- These refer to the tools, schedules, and methodologies employed in the project.

I. Use of Modern Programming Practices (MODP)

II. Development Tools (TOOL)

III. Required Development Schedule (SCED)

Intermediate COCOMO Model (contd…)

Cost Drivers	Ratings					
	Very Low	Low	Nominal	High	Very High	Extra High
Product attributes						
Required software reliability	0.75	0.88	1.00	1.15	1.40	
Size of application database		0.94	1.00	1.08	1.16	
Complexity of the product	0.70	0.85	1.00	1.15	1.30	1.65
Hardware attributes						
Run-time performance constraints			1.00	1.11	1.30	1.66
Memory constraints			1.00	1.06	1.21	1.56
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30	
Required turnabout time		0.87	1.00	1.07	1.15	
Personnel attributes						
Analyst capability	1.46	1.19	1.00	0.86	0.71	
Applications experience	1.29	1.13	1.00	0.91	0.82	
Software engineer capability	1.42	1.17	1.00	0.86	0.70	
Virtual machine experience	1.21	1.10	1.00	0.90		
Programming language experience	1.14	1.07	1.00	0.95		
Project attributes						
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82	
Use of software tools	1.24	1.10	1.00	0.91	0.83	
Required development schedule	1.23	1.08	1.00	1.04	1.10	

Intermediate COCOMO Model (contd…)

- Intermediate model equation

I. Effort(E)= $a*(KLOC)^b * EAF$ person-months

II. Development Time (T) = $c*(E)^d$ months

III. Average staff size = E/T persons

IV. Productivity = KLOC/E KLOC/PM

Where EAF= Effort Adjustment factor $EAF = \prod_{i=1}^{15} (\text{Cost Driver Multiplier for each factor})$

Software Projects	a	b	c	d
Organic	3.2	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Detailed COCOMO Model

- The **Detailed COCOMO (Constructive Cost Model)** is the most comprehensive and advanced version of the COCOMO suite.
- It extends the Intermediate COCOMO by considering project components and phases in greater detail, allowing for precise estimation of effort and cost for each part of the project lifecycle.
- Detailed cocomo model equation:

  $Effort_{detailed} = \mu p * Effort_{intermediate}$
  $Development\ time_{detailed} = tp * Development\ time_{intermediate}$

Detailed COCOMO Model (contd…)

Detailed COCOMO Model

Table 4.4: Phase-wise distribution of the development effort and time

Project type and size	Plan and requirement	System design	Detailed design	Code and unit test	Integration and test
Percentage-wise distribution of the development effort					
Organic (2 KLOC)	6	16	26	42	16
Organic (32 KLOC)	6	16	24	38	22
Semi-detached (32 KLOC)	7	17	25	33	25
Semi-detached (128 KLOC)	7	17	24	31	28
Embedded (128 KLOC)	8	18	25	26	31
Embedded (320 KLOC)	8	18	24	24	34
Percentage-wise distribution of the development time					
Organic (2 KLOC)	10	19	24	39	18
Organic (32 KLOC)	12	19	21	34	26
Semi-detached (32 KLOC)	20	26	21	27	26
Semi-detached (128 KLOC)	22	27	19	25	29
Embedded (128 KLOC)	36	36	18	18	28
Embedded (320 KLOC)	40	38	16	16	30

121

Function Point(FP)

- The function point (FP) metric can be used effectively as a means for measuring the functionality delivered by a system.
- Using historical data, the FP metric can be used to:
  - Estimate the cost or effort required to design, code and test the software,
  - Predicate the number of errors that will be encounter during testing,
  - Forecast the number of components in implemented system.
- Function Point(FP): UFP*CAF

  Where, UFP=Unadjusted Function Point and CAF=Complexity Adjustment Factor
- A system has 5 types of function units:

I. External inputs (EI):
   - External Inputs represent the data or control information the system receives from outside entities, such as users or external systems. These inputs often trigger internal processes or modify stored data. For example, when a user fills out a login form with a username and password, these inputs are sent to the system to authenticate the user.

Function Point(FP) (contd…)

II. External Output (EO):
- External Outputs refer to information or results generated by the system and sent to an external entity, typically as a response or report. These outputs may be the result of computations, formatting, or data extraction processes. For instance, after an online purchase, a system might generate an invoice summarizing the transaction details and send it to the customer via email.

III. External Inquiries (EQ):
- External Inquiries are user-initiated requests that retrieve and present data without significant transformations or processing. These queries often combine internal data from the system with external sources to display requested information. For example, when a customer searches for a product in an e-commerce store by entering a keyword, the system retrieves and displays a list of matching items.

Function Point(FP) (contd…)

IV. Internal Logical File (ILF):

- Internal Logical Files are collections of user-identifiable data or control information stored and managed internally by the system. These files represent persistent data that the system creates, updates, and deletes as part of its operations. For instance, a customer database that stores names, addresses, and order histories is an ILF because it is maintained internally by the system.

V. External Interface File (EIF):

- External Interface Files are data sets or control information that the system references but does not manage. These files are maintained by external systems and used as input or lookup references in the analyzed system. For example, an e-commerce system might use a supplier's product catalog (stored externally) to fetch product details during a purchase order. Similarly, a system may use a list of postal codes maintained by a government agency for validating addresses.

Function Point(FP) (contd…)

- Function units with weighting factors

Functional Units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
External logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Table 1

Function Point(FP) (contd…)

- The procedure for the calculation of UFP in mathematical form is given below:

$$UFP = \sum_{i=1}^{5} \sum_{J=1}^{3} Z_{ij} w_{ij}$$

- Where i indicate the row and j indicate the column of table 1

  W_{ij} = It is the entry of the i^{th} row and j^{th} column of the table 1

  Z_{ij} = It is the count of the number of function units of type i that have been classified as having the complexity corresponding to column j.

Function Point(FP) (contd…)

- The procedure for the calculation of CAF in mathematical form is given below:

$$CAF=[0.65+0.01*\sum F_i]$$

Where, the F_i (i=1 to 14) are the degree of influence (no influence = 0, incidental = 1, moderate = 2, average = 3, significant = 4, essential = 5) and are based on responses to 14 questions given below:

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

COCOMO 2

- **COCOMO II** (Constructive Cost Model II) is an updated version of the original COCOMO model developed by Barry Boehm.
- It is a software cost estimation model that helps predict the effort, time, and cost required to develop a software product.
- COCOMO II is designed to address modern software development practices and environments.
- There are types of COCOMO 2 models. They are:
  I. Application Composition Model
  II. Early Design Model
  III. Post-Architecture Model

COCOMO 2 (contd…)

Application Composition Model

- The **Application Composition Model** is one of the sub-models of **COCOMO II**, designed for the early stages of software development, specifically for systems developed using **rapid application development (RAD)** techniques or **prototyping**.
- This model is particularly suited for projects where software is assembled from pre-existing components or generated automatically using application generators.
- In this model, size is first estimated using Object Points.
- Object Points define screen, reports, and third-generation (3GL) modules as objects.

COCOMO 2 (contd…)

Steps in Application Composition Model are:

1. Access Object Counts

- Estimate the number of screens, reports and 3GL components that will comprise by application.

2. Classify Complexity Levels of Each Object

- We have to classify each object instance into simple, medium and difficult complexity level depending on values of its characteristics. Complexity levels are assigned according to the given table.

No. of views contain	Sources of data tables		
	Total < 4 (< 2 servers < 3 clients)	Total < 8 (2 - 3 servers 3-5 clients)	Total 8 + (> 3 servers > 5 clients)
< 3	Simple	Simple	Medium
3 - 7	Simple	Medium	Difficult
> 8	Medium	Difficult	Difficult

For Screens

No. of section contain	Sources of data tables		
	Total < 4 (< 2 servers < 3 clients)	Total < 8 (2 - 3 servers 3-5 clients)	Total 8 + (> 3 servers > 5 clients)
0 - 1	Simple	Simple	Medium
2 - 3	Simple	Medium	Difficult
4 +	Medium	Difficult	Difficult

For Reports

COCOMO 2 (contd…)

3. Assign Complexity Weights to Each Object

- The weights are used for three object types i.e, screens, reports and 3GL components. Complexity weight are assigned according to object's complexity level using following table.

Object Type	Complexity Weight		
	Simple	Medium	Difficult
Screen	1	2	3
Report	2	5	8
3GL Components	-	-	10

Complexity Weight

COCOMO 2 (contd…)

4. Determine Object Points

- Add all the weighted object instances to get one number and this is known as object point count.
- Object point = Σ (number of object instances) *(Complexity weight of each object instance)

5. Compute New Object Points (NOP)

- We have to estimate the %reuse to be achieved in a project. Depending on %reuse, NOP are the object point that will need to be developed and differ from the object point count because there may be reuse of some object instance in project.
- *NOP = [(object points) * (100 – % reuse)] / 100*

COCOMO 2 (contd…)

6. Calculate Productivity rate (PROD)

- Productivity rate is calculated on the basis of information given about developer's experience and capability. For calculating it, we use following table.

Developers experience & capability	Productivity (PROD)
Very Low	4
Low	7
Nominal	13
High	25
High	50

Productivity Rate

COCOMO 2 (contd…)

7. Compute the estimated Effort

- Effort to develop a project can be calculated as:
- *Effort = NOP / PROD*

COCOMO 2 (contd…)

Early Design Model

- The **Early Design Model** in **COCOMO II** is a cost estimation model used during the early stages of software development, when the software system's architecture is not fully detailed, but high-level design decisions are made.
- It provides a rough estimate of the effort required based on initial inputs like size and general project characteristics.

$$PM_{nominal} = A * (Size)^{B}$$

Where

PM*nominal* = Effort of the project in person months
A = Constant representing the nominal productivity, provisionally set to 2.5
B = Scale factor
Size = Software size

COCOMO 2 (contd…)

The **Scaling Factors** that COCOMO-II model uses for the calculation of B are:

- Precedentness **(PREC)**
- Development flexibility **(FLEX)**
- Architecture/ Risk Resolution **(RESL)**
- Team Cohesion **(TEAM)**
- Process maturity **(PMAT)**

Data for the Computation of B (Scalar Factor)

Scaling Factors	Very Low	Low	Nominal	High	Very High	Extra High
Precedentness	6.20	4.96	3.72	2.48	1.24	0.00
Development Flexibility	5.07	4.05	3.04	2.03	2.03	0.00
Architecture/ Risk Resolution	7.07	5.65	4.24	2.83	1.41	0.00
Team Cohesion	5.48	4.38	3.29	2.19	1.10	0.00
Process Maturity	7.80	6.24	4.68	3.12	1.56	0.00

COCOMO 2 (contd…)

Early design cost drivers

There are 7 early design cost drivers and are given below:

1. Product Reliability and Complexity **(RCPX)**
2. Required Reuse **(RUSE)**
3. Platform Difficulty **(PDIF)**
4. Personnel Capability **(PERS)**
5. Personnel Experience **(PREX)**
6. Facilities **(FCIL)**
7. Schedule **(SCED)**

Early Design Cost Drivers	Extra Low	Very Low	Low	Nominal	High	Very High	Extra High
RCPX	0.73	0.81	0.98	1.0	1.30	1.74	2.38
RUSE	-	-	0.95	1.0	1.07	1.15	1.24
PDIF	-	-	0.87	1.0	**1.29**	1.81	2.61
PERS	2.12	1.62	1.26	1.0	**0.83**	0.63	0.50
PREX	1.59	1.33	1.12	1.0	0.87	0.71	0.62
FCIL	1.43	1.30	1.10	1.0	0.87	0.73	0.62
SCED	-	1.43	1.14	1.0	1.0	1.0	-

COCOMO 2 (contd…)

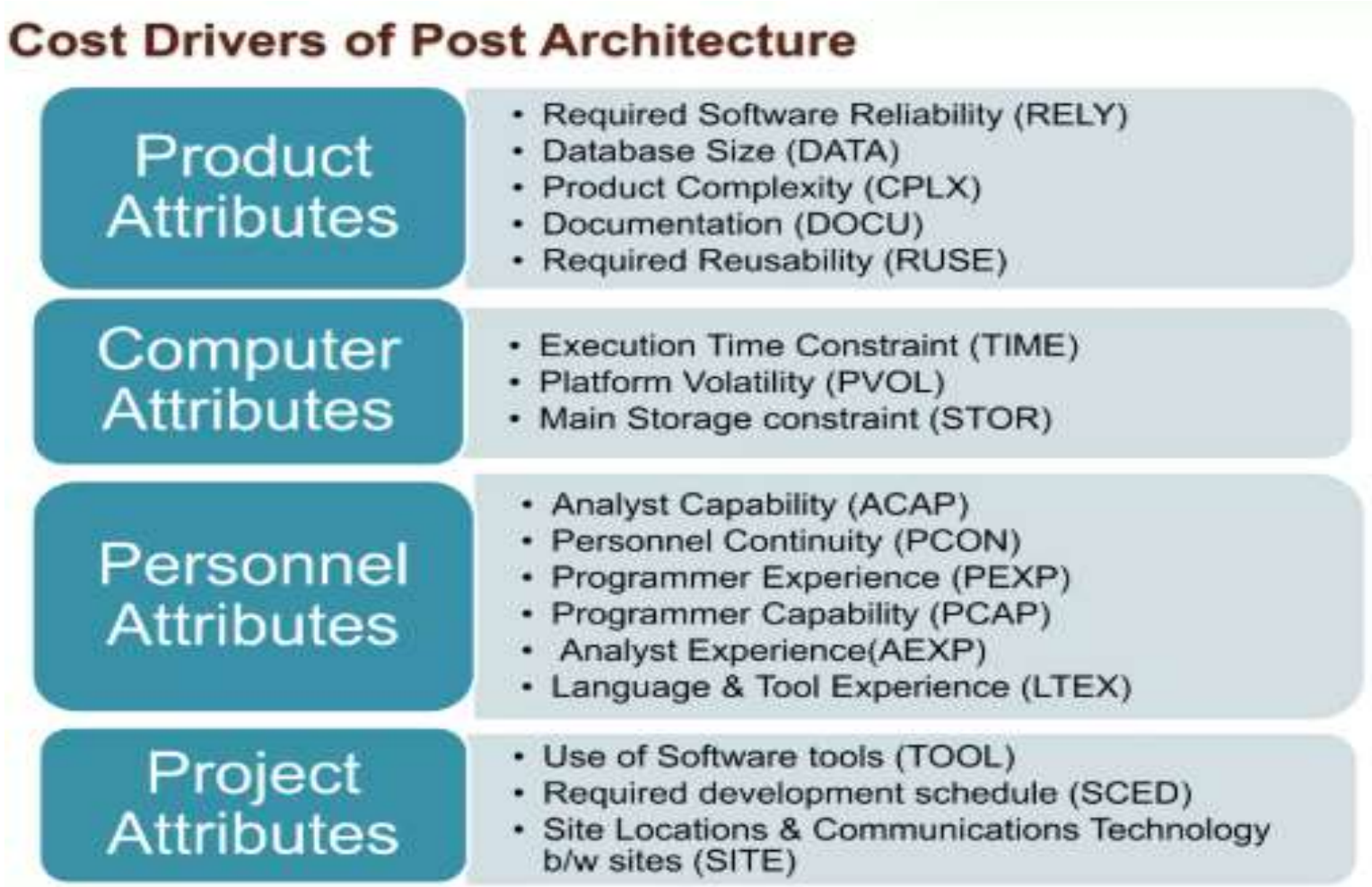
$$PM_{adjusted} = PM_{nominal} \times \left[\prod_{i=1}^{7} EM_i \right]$$

COCOMO 2 (contd…)

Post-Architecture Model

- The **Post-Architecture Model** is the most detailed and comprehensive model of **COCOMO II**, used for estimating the effort required during the later stages of software development. It is applied when the software architecture has been defined and detailed design activities have commenced.
- This model is suitable for projects with a well-understood architecture, allowing for precise estimates of effort and schedule.

COCOMO 2 (contd…)



COCOMO 2 (contd…)

17 Cost Drivers

Cost Drivers	Very Low	Low	Nominal	High	Very High	Extra High
RELY	0.75	0.88	1.00	2.48	1.24	0.00
DATA		0.93	1.00	2.03	2.03	0.00
CPLX	0.75	0.88	1.00	2.83	1.41	0.00
RUSE		0.91	1.00	2.19	1.10	0.00
DOCU	0.89	0.95	1.00	3.12	1.56	0.00
TIME			1.00	1.11	1.31	1.67
STOR			1.00	1.06	1.21	1.57
PVOL		0.87	1.00	1.15	1.30	
ACAP	1.50	1.22	1.00	0.83	0.67	
PCAP	1.37	1.16	1.00	0.87	0.74	
PCON	1.24	1.10	1.00	0.92	0.84	
AEXP	1.22	1.10	1.00	0.89	0.81	
PEXP	1.25	1.12	1.00	0.88	0.81	
LTEX	1.22	1.10	1.00	0.91	0.84	
TOOL	1.24	1.12	1.00	0.86	0.72	
SITE	1.25	1.10	1.00	0.92	0.84	0.78
SCED	1.29	1.10	1.00	1.00	1.00	

COCOMO 2 (contd…)

$$PM_{adjusted} = PM_{nominal} \times \left[\prod_{i=1}^{17} EM_i \right]$$

Software Risks

- Risk is uncertain events associated with future events that have a probability of occurrence but it may or may not occur and if occurs it brings loss to the project.
- Risk identification and management are very important tasks during software project development because the success and failure of any software project depend on it.
- **Software Risks** refer to potential issues that may negatively impact the success of a software project, leading to delays, increased costs, or failure to meet user requirements.

Various Kinds of Risks in software are:

I. Schedule Risk:

- Schedule related risks refers to time related risks or project delivery related planning risks.
- The wrong schedule affects the project development and delivery.
- These risks are mainly indicates to running behind time as a result project development doesn't progress timely and it directly impacts to delivery of project.
- Finally if schedule risks are not managed properly it gives rise to project failure and at last it affect to organization/company economy very badly.

Software Risks (contd….)

Some reasons for Schedule risks:

- Time is not estimated perfectly
- Improper resource allocation
- Tracking of resources like system, skill, staff etc
- Frequent project scope expansion
- Failure in function identification and its' completion

Software Risks (contd…)

II. Budget Risk:

- Budget related risks refers to the monetary risks mainly it occurs due to budget overruns.
- Always the financial aspect for the project should be managed as per decided but if financial aspect of project mismanaged then there budget concerns will arise by giving rise to budget risks.
- So proper finance distribution and management are required for the success of project otherwise it may lead to project failure.

Some reasons for Budget risks:

- Wrong/Improper budget estimation
- Unexpected Project Scope expansion
- Mismanagement in budget handling
- Improper tracking of Budget

Software Risks (contd…)

III. Operational Risk:

- Operational risk refers to the procedural risks means these are the risks which happen in day-to-day operational activities during project development due to improper process implementation or some external operational risks.

Some reasons for Operational risks:

- Insufficient resources
- Improper management of tasks
- Less number of skilled people
- Lack of communication and cooperation
- Lack of clarity in roles and responsibilities
- Insufficient training

Software Risks (contd…)

IV. Technical Risk:

- Technical risks refers to the functional risk or performance risk which means this technical risk mainly associated with functionality of product or performance part of the software product.
- Technical risks identify potential design, implementation, interface, verification, and maintenance problems.

Some reasons for Technical risks

- Frequent changes in requirement
- Less use of future technologies
- Less number of skilled employee
- High complexity in implementation
- Improper integration of modules

Software Risks (contd…)

V. Business Risk:

- It includes the top four business risks which are

i. Market risk: system that no one really wants

ii. Strategic risk: product no longer fits into the overall business strategy

iii. Sales risk: the sales force doesn't understand how to sell the product

iv. Management risk: losing the support of senior management

VI. Unpredictable Risk:

- They can and do occur, but they are extremely difficult to identify in advance.

Software Assumption

- **Software Assumptions** refer to the preconditions or expectations made during software development about the environment, inputs, or behavior of the system being built.
- These assumptions help simplify the design and development process, but they must be well-documented and validated to avoid risks or failures.

Importance of Software Assumption

- **Foundation for Planning**: Assumptions make the basis for the planning of the project. They serve as pillars that the project plans rest upon. They serve as the setting and provide the surrounding situations where the proposed project is to take place.
- **Risk Identification**: Assumptions demonstrate the downside of the case study and the inaccurate issues. A smart thing project managers can do is to document and figure out the assumptions they make. They can thus assess the possible risks that come from those assumptions and start working on solutions to eliminate the risks.

Software Assumption (contd…)

- **Decision-Making**: Assumptions guide decision-making processes. The project manager and team members use assumptions as the basis for making such crucial decisions as resource allocation, scheduling, and others, while still handling multiple logical aspects of project management planning and implementation.
- **Communication**: Communication plays a central role in project achievement of a project. The assumption is the basis of achievement if one is documented and communicated to all stakeholders to create an obvious understanding of the truth that surrounds the entire project. This is a great way for the exchange across cultures to continue which will eliminate the possibility of confusion.
- **Scope Definition**: Assumptions play an important role in determining the extent or the extent of the project task. They are of help in defining the delimitations and limitations that will bind the project team to operate within.
- **Risk Management**: Assumptions and risks are deeply related as risks are formed based on assumptions. The failure in assumption handling can put you into serious preventable risks. Therefore, it is always necessary to outline risk management strategies like making contingency plans to avoid these risks.

Software Assumption (contd…)

- **Continuous Monitoring**: A project lifecycle is an interactive process; assumptions should be kept an eye on and if proven inaccurate checked and corrected. Nonetheless, this process never stops, and that is the main reason why the project team stays notified about any changes in the conditions or environment that may influence the project's achievements.

Software Issues (contd…)

- **Software issues** refer to challenges or problems that arise during the development, deployment, and maintenance of software systems.
- These issues can affect the functionality, performance, reliability, or usability of the software.
- Common software issues are:

I. **Software that is difficult to use** - Many people have experienced first-hand the frustration of using software that is cumbersome, difficult to navigate, and requires several steps to perform simple tasks. This problem relates to a lack of understanding of how humans interact with computers and is also the result of a history of modifications that are not planned and coordinated to account for ease of use.

Software Issues (contd…)

II. **Inconsistent processing** - Software that only works correctly in one environment – This refers to software that has been designed for only one environment and cannot be easily transported and used in another environment. Of course, some software is designed to work in only one environment. However, if an organization adopts new technology that requires software be portable to new environments, then the software will need to be modified or replaced if it can't meet the new technical requirements. An example of this is software that works in an MS-DOS environment, but will not work in a Microsoft Windows environment.

III. **Difficult to maintain and understand** - This refers to the ability of a programmer or developer to maintain the software. To maintain software, the person performing the maintenance must first analyze and understand the software. Much of the software in existence today was initially written in an unstructured manner and then patched on an as-needed basic over a long period of time. This type of software structure results in what is known as "spaghetti code," which is complex and unstructured. To add to the problem, when changes are made to this kind of software, there is a higher risk of creating new defects unintentionally.

Software Issues (contd…)

IV. **Unreliable results or performance** - This means that the software does not deliver consistently correct results or cannot be depended to work correctly each time it is used.

V. **Inadequate support of business needs or objectives** - This refers to software that is inflexible to meeting business needs. For example, a system may be difficult to modify to meet and organization's needs or may lack features to allow the users to customize business rules.

VI. **No longer supported by the vendor** - This occurs when a vendor ceases to support a particular software product. This can occur due to the vendor's decision to no longer support a product, due to the vendor going out of business, or the vendor selling the product to another vendor.

VII. **Incorrect or inadequate interfaces with other systems** - This means that the software does not correctly accept input (data, control, parameters, etc.) from other systems or sends incorrect output (data, control, parameters, print, etc.) to other systems.

Software Issues (contd…)

VIII. **Inadequate security controls** - This means that unauthorized access to the system is not adequately controlled and detected. In addition, people may also be able to perform transactions in excess of the authorization levels appropriate for their job functions.

Software Issues (contd…)

Common software issues categorized for clarity:

I. **Development-Related Issues**

- These issues occur during the software development phase:
- **Requirement Ambiguity**: Poorly defined or misunderstood requirements can lead to incorrect or incomplete software functionality.
- **Scope Creep**: Uncontrolled changes or continuous addition of new features can derail the project timeline and budget.
- **Code Quality**: Poor coding practices lead to bugs, inefficiencies, and maintainability challenges.
- **Integration Problems**: Difficulty in integrating different software components or third-party tools.

Software Issues (contd…)

II. **Testing and Quality Assurance Issues**

- These arise during the testing and validation phase:
- **Insufficient Testing**: Missing test cases or inadequate testing can leave critical bugs unnoticed.
- **Environment Mismatch**: Testing in environments that don’t match the production environment leads to undetected issues.
- **Performance Bottlenecks**: Software fails under heavy load or does not meet speed expectations.
- **Regression Issues**: Fixing one bug unintentionally creates new ones.

Software Issues (contd…)

III. **User-Related Issues**

- Problems experienced by the end-users:
- **User Experience (UX) Problems**: Poor interface design makes the software hard to use.
- **Functionality Mismatch**: Features don’t meet user expectations or are difficult to access.
- **Inadequate Documentation**: Lack of clear instructions for users to operate the software effectively.

Software Issues (contd…)

IV. **Maintenance Issues**

- Challenges faced during software updates or bug fixes:
- **Technical Debt**: Accumulated shortcuts in development make maintenance harder.
- **Backward Compatibility**: Updates break functionality in older versions.
- **Security Vulnerabilities**: Bugs or outdated components leave the software open to attacks

V. **Security Issues**

- Threats that compromise the confidentiality, integrity, or availability of software:
- **Data Breaches**: Unauthorized access to sensitive information.
- **Malware Attacks**: Software is exploited to deliver malicious code.
- **Weak Authentication**: Poor password policies or lack of multi-factor authentication.

Software Issues (contd…)

VI. **Project Management Issues**

- Challenges in managing the software development lifecycle:
- **Missed Deadlines**: Failure to deliver the software on time.
- **Budget Overruns**: Costs exceeding initial estimates.
- **Poor Communication**: Lack of coordination among team members.

VII. **Compatibility Issues**

- Problems when integrating with other software or systems:
- **Platform Dependence**: Software runs only on specific hardware or OS.
- **API Incompatibility**: Updates to third-party APIs break existing functionality.

Software Dependency

- **Software dependency** refers to the reliance of one software component, application, or system on another to function correctly.
- Dependencies can include libraries, frameworks, APIs, external services, or even hardware-specific components.
- Managing dependencies is critical in software development to ensure functionality, maintainability, and security.

Types of software dependency

I. Direct software dependency

II. Transitive software dependency

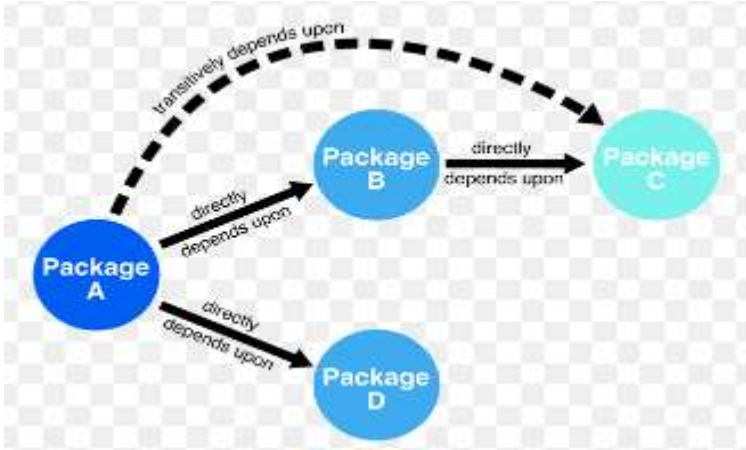
Software Dependency (contd…)

Direct Software Dependency:

- A **direct dependency** occurs when a software application explicitly includes or relies on a specific library, framework, or external resource to function correctly.
- These dependencies are immediately visible in the application's configuration or code, and the developer intentionally includes them in the project.

Transitive Software Dependency:

- A **transitive dependency** occurs indirectly when a direct dependency itself depends on another library or module.
- Transitive dependencies are not explicitly included by the developer but are automatically brought in by the direct dependency.



Risks Identification

- **Risk Identification** is the process of recognizing potential problems, uncertainties, or threats that could negatively impact a software project.
- Identifying risks early allows project managers and teams to proactively plan strategies to mitigate or eliminate their impact.
- The different categories of risk (project, technical, business, known, predictable, and unpredictable) can be further divided as:

I. Generic risks: They are a potential threat to every software project.

II. Product-specific risks: They can be identified only by those with a clear understanding of the technology, the people, and the environment that is specific to the software that is to be built. To identify product-specific risks, the project plan and the software statement of scope are examined, and analyze the special characteristics of any product that may threaten the project plan.

Risks Identification (contd…)

Techniques for risks identification:

Checklist Analysis – Checklist Analysis is type of technique generally used to identify or find risks and manage it. The checklist is basically developed by listing items, steps, or even tasks and is then further analyzed against criteria to just identify and determine if procedure is completed correctly or not. It is list of risk that is just found to occur regularly in development of software project.

Brainstorming – This technique provides and gives free and open approach that usually encourages each and everyone on project team to participate. It also results in greater sense of ownership of project risk, and team generally committed to managing risk for given time period of project. It is creative and unique technique to gather risks spontaneously by team members. The team members identify and determine risks in ‘no wrong answer’ environment. This technique also provides opportunity for team members to always develop on each other’s ideas. This technique is also used to determine best possible solution to problems and issue that arises and emerge.

Risks Identification (contd…)

SWOT Analysis – Strengths-Weaknesses-Opportunities-Threat (SWOT) is very technique and helpful for identifying risks within greater organization context. It is generally used as planning tool for analyzing business, its resources, and also its environment simply by looking at internal strengths and weaknesses and opportunities and threats in external environment. It is technique often used in formulation of strategy.

Flowchart Method – This method allows for dynamic process to be diagrammatically represented in paper. This method is generally used to represent activities of process graphically and sequentially to simply identify the risk.

Risks Identification (contd…)

One method for identifying risks is to create a risk item checklist; we need to identify the following area from where risk will appear:

- Product size—risks associated with the overall size of the software to be built or modified.
- Business impact—risks associated with constraints imposed by management or the marketplace.
- Stakeholder characteristics—risks associated with the sophistication of the stakeholders and the developer's ability to communicate with stakeholders in a timely manner.
- Process definition—risks associated with the degree to which the software process has been defined and is followed by the development organization.
- Development environment—risks associated with the availability and quality of the tools to be used to build the product.
- Technology to be built—risks associated with the complexity of the system to be built and the "newness" of the technology that is packaged by the system.
- Staff size and experience—risks associated with the overall technical and project experience of the software engineers who will do the work.

Risks Identification (contd…)

The risk components defined by U.S. air force are defined in the following manner:

- Performance risk—the degree of uncertainty that the product will meet its requirements and be fit for its intended use.
- Cost risk—the degree of uncertainty that the project budget will be maintained.
- Support risk—the degree of uncertainty that the resultant software will be easy to correct, adapt, and enhance.
- Schedule risk—the degree of uncertainty that the project schedule will be maintained and that the product will be delivered on time.

The impact of each risk component is divided into one of four categories—negligible, marginal, critical, and catastrophic.

Risks Identification (contd…)

Risk Projection

- Risk Projection attempts to rate each risk by calculating the probability that the risk is real and the consequences of the problems associated with the risk, should it occur.
- During risk estimation, the first step is the prioritizing risks and hence we can allocate resources where they will have the most impact.
- The process of categorization of various possible risks that may appears in in project is called risk assessment.
- The analysis of nature, scope and time are main component of risk assessment.

Risks Identification (contd…)

Risk Table

- A risk table provides a simple technique for risk projection. The risk table can be implemented as a spreadsheet model.
- This enables easy manipulation and sorting of the entries.

Risks Identification (contd…)

Risks	Category	Probability	Impact	RMMM
Size estimate may be significantly low	PS	60%	2	
Larger number of users than planned	PS	30%	3	
Less reuse than planned	PS	70%	2	
End-users resist system	BU	40%	3	
Delivery deadline will be tightened	BU	50%	2	
Funding will be lost	CU	40%	1	
Customer will change requirements	PS	80%	2	
Technology will not meet expectations	TE	30%	1	
Lack of training on tools	DE	80%	3	
Staff inexperienced	ST	30%	2	
Staff turnover will be high	ST	60%	2	
MMM				

Impact values:
1—catastrophic
2—critical
3—marginal
4—negligible

Risk Category:
PS = Project size risk
BU = Business risk
TE = Technology risk
DE = Development risk
ST = Stakeholder risk
CU =Customer risk

Fig. RISK TABLE

- Here, at first all risks are listed in the first column of the table. This can be accomplished with the help of the risk item checklists reference. Each risk is categorized in the second column. The probability of occurrence of each risk is entered in the next column of the table. The probability value for each risk can be estimated by team members individually.

Risks Identification (contd…)

- Finally, the table is sorted by probability and by impact. High-probability, high-impact risks percolate to the top of the table, and low-probability risks drop to the bottom.
- The column labeled RMMM contains a pointer into a risk mitigation, monitoring, and management plan.
- Risk impact and probability have a distinct influence on management concern.
- A risk factor that has a high impact but a very low probability of occurrence should not absorb a significant amount of management time.
- However, high-impact risks with moderate to high probability and low-impact risks with high probability should be carried forward into the risk analysis steps that follow.

Risks Handling Strategies

There are two ways that software engineers can handle risk:

Reactive risk strategies: Software engineer corrects a problem as it occurs or never worrying about problems until they happened. This is often called a fire-fighting mode.

Proactive risk strategies: A considerably more intelligent strategy for risk management where a proactive software engineer starts thinking about possible risks in a project before they occur. A proactive strategy begins long before technical work is initiated where potential risks are identified, their probability and impact are assessed, and they are ranked by importance. Then, the software team establishes a plan for managing risk. The primary objective is to avoid risk, but all risks cannot be avoided so exercise to make them minimal.

Thank You